

# FastEvo: Accelerating Reward Evolution with Multi-Agent Evaluation for Robotic Manipulation

Anonymous CVPR submission

Paper ID 8389

## Abstract

We introduce *FastEvo*, a novel framework for accelerating the evolution of LLM-generated reward functions in robotic policy learning. While recent work shows that LLM-generated reward functions can effectively replace human-crafted ones by being refined through environmental feedback iteratively, the evolution of such rewards remains constrained by two key bottlenecks: imprecise localization of policy errors and insufficient failure analysis to guide LLM-based refinement. To address these challenges, *FastEvo* first trains a language-conditioned scoring model to accurately identify failure onsets in policy execution videos. The corresponding video frames are then processed by a query-analysis-diagnosis multi-agent system, which conducts in-depth policy failure analysis. Finally, *FastEvo* delivers a detailed report with constructive suggestions to facilitate reward function evolution. Experimental results demonstrate that *FastEvo* achieves 78% accuracy in locating error onsets and enables efficient reward evolution to train a policy with a 94% success rate within only 10 iterations, representing a significant advance in speeding up reward function evolution for robotic policy learning.

## 1. Introduction

Reinforcement learning (RL) has achieved impressive results in robotic policy learning, enabling high success rates and generalizability through carefully designed reward functions [2, 12]. However, The design of effective reward functions is notoriously difficult [36, 41], typically requiring substantial human labor and expert knowledge.

Fortunately, the development of simulation platforms such as Isaac Gym [29] has enabled online reinforcement learning with programmatically generated reward functions, establishing LLM-generated rewards as a promising alternative. Leveraging their capabilities in code generation, zero-shot reasoning, and in-context learning, LLMs can dynamically create and refine reward functions within simu-

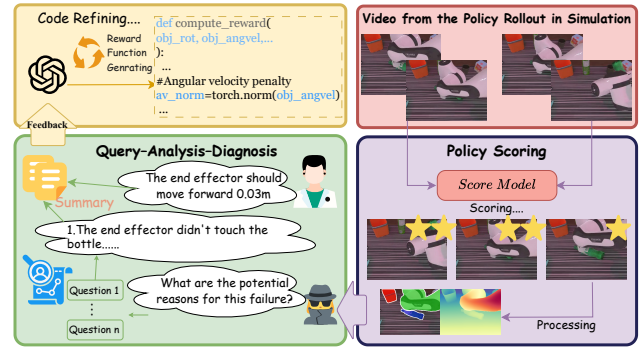


Figure 1. **Overview of FastEvo**. FastEvo is a closed-loop framework that accelerates the evolution of LLM-generated reward functions for robotic policy learning. It leverages a pre-trained scoring model and a multi-agent query-analysis-diagnosis system to achieve precise error localization and in-depth policy failure evaluation.

lated environments [27, 44, 47, 53]. Nevertheless, when LLMs generate flawed reward code, they can misdirect the learning process toward suboptimal policies and degrade robot performance. Though incorporating humans into the design loop can help steer reward functions toward better solutions, recent studies reveal that exposing humans to incorrect AI suggestions prior to decision-making increases their likelihood of selecting wrong answers, even in situations where they would otherwise have been correct [8]. To address this limitation, recent studies [6, 22, 33, 57] position VLMs as teachers that identify policy failures during execution and guide LLM students through self-reflection to refine reward functions. However, the capacity of a single VLM remains insufficient for both accurate failure localization and in-depth failure analysis. This limitation hinders effective guidance for LLM-based reward refinement and ultimately constrains the evolution of the reward function.

Given the limited ability of a single VLM, it is natural to consider incorporating multiple agents. Numerous studies [5, 13, 20, 40] show that multi-agent systems leverage the collaborative power of specialized agents to solve complex problems that are difficult for a single model. Yet, a

critical question remains: how should such a multi-agent system be designed?

Based on the educational principle that even less gifted students can accomplish complex tasks with proper guidance, we propose FastEvo, a novel step-by-step framework structured around a query-analysis-diagnosis multi-agent system under the guidance of a pretrained scoring model. We first train the scoring model for execution error localization using a dataset composed of Open-X trajectories [31] and failure cases collected by DP3 [54]. Then the model assigns a score to each frame of a policy execution video and identifies the frame with the lowest score as the error onset. Once the error onset is found, the corresponding video frames are processed by our Query-Analysis-Diagnosis Multi-Agent System to perform in-depth failure analysis. In this process, the query agent identifies potential failure modes and raises diagnostic questions based on the visual input. The Spatial Information Extraction Pipeline then extracts relative spatial information from the video frame and prompts the Analysis Agent to answer those questions with self-voting mechanism. Finally, the Diagnosis Agent summarizes the answers to provide constructive suggestions that directly inform reward function evolution.

Our approach demonstrates superior performance in comparisons with all relevant baselines. It achieves 78% accuracy in locating error onsets and enables efficient reward evolution to train a policy with a 94% success rate within only 10 iterations, representing a significant advance in speeding up reward function evolution for robotic policy learning. Furthermore, we conduct ablation studies on key components of FastEvo and find that: (1) Task instructions and demonstration decomposition significantly enhance the scoring model's error localization capability; (2) The efficiency of reward function evolution exhibits a positive correlation with the quality of queries generated by the query agent; and (3) Spatial information substantially improves the stability of the reward function evolution process. Our contributions as follows:

- We introduce FastEvo, a novel framework that accelerates reward function evolution for robotic policy learning through a query-analysis-diagnosis multi-agent system guided by a pretrained scoring model. Our approach will be fully open-sourced.
- We validate our approach by comparing it with all the relevant baselines, demonstrating it achieves 24% higher error localization accuracy and faster reward evolution than the best baseline.
- Our comprehensive analysis of FastEvo offers foundational insights, paving the way for future advancements in policy error localization and failure evaluation with LLM-based agents.

## 2. Related Works

### 2.1. Policy Learning via LLMs

The growing reasoning and code generation capabilities of LLMs have made them powerful tools for robotic policy learning. Early works [14, 15, 23, 24, 38, 43] focused on using LLMs to translate plain instructions into structured code for specific robotic tasks, but these approaches have limited generalizability due to their reliance on predefined primitives. Then subsequent works [47, 53] employed LLMs to generate reward functions for reinforcement learning but still required costly human refinement. More recent works [17, 27, 44, 51, 53] leverages LLMs' self-correction capability to enable autonomous reward function evolution. However, slow convergence persists due to inefficient error localization during robotic execution. To solve this problem, FastEvo employs a scoring model for precise error localization during robotic policy execution, accelerating reward function evolution.

### 2.2. Policy Evaluation

Comprehensive policy evaluation has become significant as robotic policies advance. Traditional approaches rely on physical deployment, suffering from high costs, slow iteration, and poor scalability. So subsequent works have evolved along two primary directions. One employs standardized benchmarks [4, 11, 19, 25] that offer common objects and tasks, while the other utilizes remote access solutions [10, 46, 48] that enable remote robot operation. Nevertheless, both approaches still require specific hardware and result in significant costs. Consequently, simulation-based evaluation has emerged [3, 21, 50], yet most implementations rely on handcrafted evaluation metrics. Recent approaches further leverage Vision-Language Models (VLMs) for evaluation [6, 22, 33, 57], but they only look at surface-level problems and can't identify the underlying causes of failures. In contrast, FastEvo enables a comprehensive analysis of policy failures through a query-analysis-diagnosis multi-agent system.

### 2.3. Multi-Agent System

A multi-agent system comprises multiple autonomous agents, each perceiving environments, making decisions, and executing actions [7]. This architecture enables specialized agents to collectively achieve complex goals unattainable by single models [5, 13, 20, 40]. Recent works have demonstrated that multi-agent systems exhibit significant capabilities across complex domains, including software development [32, 45], medicine [18] and embodied intelligence [16, 42]. In Embodied AI, ReAd [56] attempts to speed up LLM-based reward function evolution by integrating advantage functions with LLM planning, yet solely focus on subtask outcomes while neglecting process dynam-

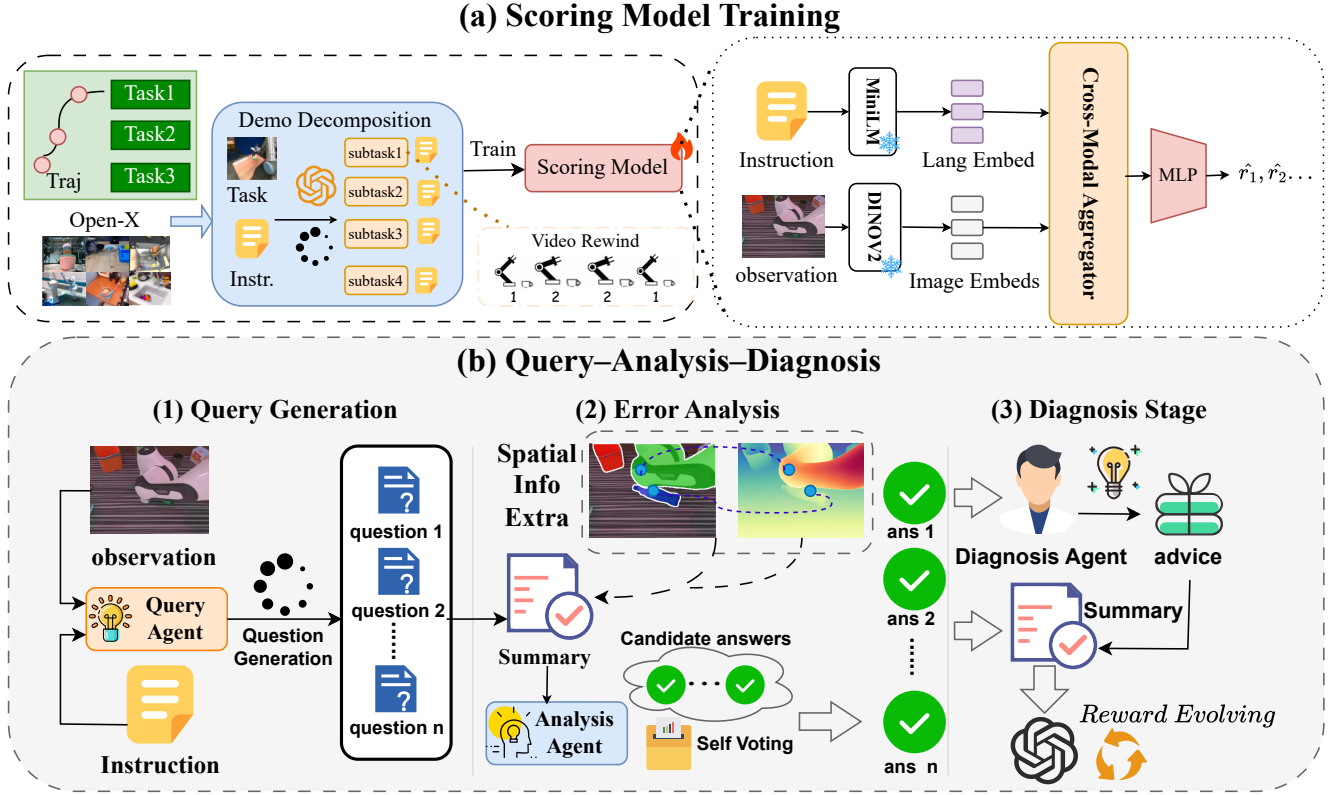


Figure 2. **Architecture Overview.** (a) **Scoring Model Training:** We train our Scoring Model on a dataset constructed from the Open-X dataset and DP3-collected trajectories, augmented with video rewinding, demonstration decomposition, and instruction generation. The model assigns a score to each frame of a policy execution video and identifies the frame with the lowest score as the error onset. (b) **Query-Analysis-Diagnosis Multi-Agent System:** It starts with the Query Agent identifying potential failure causes and formulating diagnostic questions. These questions are then answered by the Analysis Agent using detailed spatial information extracted by Spatial Information Extraction Pipeline. After that, the Diagnosis Agent brings everything together to recommend advice to fix it. The generated advice alongside the original responses subsequently accelerates the evolution of the LLM-based reward function.

ics. In contrast, FastEvo conducts comprehensive step-level analysis of robotic execution failure to accelerate reward function evolution.

### 3. Method

Our objective is to enhance error localization accuracy and enable in-depth failure analysis for robotic policy rollouts, thereby accelerating the evolution of the reward function to achieve high success rates with faster convergence. To this end, we propose **FastEvo**, which introduces a novel Query-Analysis-Diagnosis multi-agent system built upon an error localization scoring model. The overall pipeline is illustrated in Fig. 2, and the following sections detail the implementation of each component.

#### 3.1. Scoring Model Pretraining

##### 3.1.1. Learning Objective

Our goal is to pretrain a scoring model that identifies error onset through minimally scored frames. Consequently,

aligning per-frame scores with episode success rate is critical for our scoring model. Building on Rewind [55]’s demonstration of strong alignment between predicted average reward and episode success rate, we extend their reward prediction loss by incorporating a per-frame error penalty, resulting in a more fine-grained score prediction loss:

$$\mathcal{L}_{\text{score}}(o_{1:T}, z, o_{1:T}^{\text{other}}) = \sum_{t=1}^T \left( S_{\psi}(o_t, z) - \underbrace{\frac{t}{T}}_{\text{matched seq. progress}} \right)^2 + \sum_{t=1}^T (S_{\psi}(o_t, z) - \underbrace{S_{\text{error}}}_{\text{matched error score}})^2 + \sum_{t=1}^T \underbrace{S_{\psi}(o_{1:t}^{\text{other}}, z)^2}_{\text{mismatched seq. 0 progress}}$$

This objective processes an observation sequence  $o_{1:T}$  conditioned on instruction  $z$ , along with unrelated observation

sequences  $o_{1:T}^{other}$ . It minimize the difference between the frame-level score  $S_\psi(o_t, z)$  and both the sequence progress and  $S_{error}$ , where 1 denotes success frames, -1 indicates failure onset frames and 0 represents post-failure frames. Mismatched frame scores are explicitly penalized to enforce temporal alignment. This objective offers stable training targets and is transformed into a dense [-1,1] score for precise error localization.

### 3.1.2. Data Preparation

We construct our dataset by first sampling trajectories from the Open-X Dataset [31], which provides object-centric language instructions. On a randomly selected subset, we apply the video rewinding technique from Rewind [55] to simulate failure cases, annotating the transition point between forward and reverse segments as the failure onset. In addition, we deploy a DP3 [54] model across multiple simulation environments to collect failure trajectories, with their failure onsets manually annotated. To further enhance error localization capability, we perform fine-grained data augmentation using GPT-4o, which decomposes tasks into subtasks, segments videos into corresponding episodes, and generates 5–10 supplementary instructions per subtask. Eventually, the resulting dataset comprises approximately 200k trajectories.

### 3.1.3. Architecture

Our framework employs video and text encoders following the ReWind [55] architecture, which is designed for reward generation from policy rollout videos. The overall model structure is depicted in Fig. 2. First, each video frame is processed by the DINOv2 [30] encoder, while the video instruction is encoded by the all-MiniLM-L12-v2 [35] model. Then, we train a compact cross-modal sequential aggregator that utilizes only first-frame positional embeddings to directly generate the score for each frame.

## 3.2. Query-Analysis-Diagnosis Pipeline

Our Query–Analysis–Diagnosis pipeline utilizes LLM-based agents to perform reasoning for in-depth policy error analysis. As shown in Fig. 2, the pipeline operates through three core stages: query generation, error analysis, and diagnosis stage. It eventually delivers a structured diagnostic report that identifies root causes and proposes solutions, enabling the LLM to gain a deep understanding of policy shortcomings and facilitating efficient reward function refinement.

### 3.2.1. Query Generation

The Query Generation Stage leverages the reasoning capabilities of Query Agents to identify potential causes of policy failures. When provided with failure video frames and corresponding instructions, the Query Agent analyzes potential reasons for policy failure and generates candidate

problems to guide the Analysis Agent’s targeted investigation. We then employ a structured prompt to guide the agent in generating 100 questions in a coarse-to-fine manner, from global policy issues to local execution details.

### 3.2.2. Error Analysis

The Error Analysis Stage is designed to perform accurate error analysis by responding to the queries from the Query Agent (see Sec. 3.2.1). To capture comprehensive failure information and thereby generate more precise answers, this stage leverages two components: Spatial Information Extraction Pipeline and an Analysis Agent. The Spatial Information Extraction Pipeline first extracts the relative spatial relationships between the robotic end-effector and the operated objects from the video frame. We then integrate this information with the original queries to guide the Analysis Agent in reasoning toward the final answers.

**Spatial Information Extraction Pipeline** This pipeline extracts spatial information through a two-step process: first, it segments the failure frames using SAM2 [34] to generate masks of the robotic end-effector and the manipulated objects; then, it aligns these masks with the depth map produced by DepthAnything v2 [49]. These processes yield the relative spatial relationship between the robotic end-effector and the manipulated objects, which can significantly enhance the stability of the analysis agents’ answer quality, as shown in Fig. 9. It is important to note that objects in the failure frame may sometimes be occluded. In such cases, an additional frame is extracted from the post-failure sequence for segmentation to better preserve the information of the failure case.

**Analysis Agent** The Analysis Agent is designed to identify the root causes of robotic manipulation failures by leveraging queries from the Query Agent (see Sec. 3.2.1) and spatial information extracted by the Spatial Information Extraction Pipeline. Specifically, it first generates five detailed candidate answers for each query and then conducts a self-voting process among these candidates to select the highest-scoring response, thereby enhancing the accuracy of the final answer.

### 3.2.3. Diagnosis Stage

The Diagnosis Stage is built around the Diagnosis Agent. This agent leverages the analysis from the Analysis Agent (Sec. 3.2.2) to provide constructive suggestions. These suggestions are then summarized alongside the original analysis into a report, thereby speed up the evolution of the reward function for policy learning.

**Diagnosis Agent** The Diagnosis Agent is designed to imitate expert reasoning in generating corrective suggestions and structuring the final diagnostic report. Specifically, upon receiving the initial analysis, the Diagnosis Agent identifies the primary cause of failure and formulates poten-



tial solutions, ultimately delivering a structured report with actionable recommendations to speed up the evolution of the reward function for policy learning.

### 3.2.4. Overall System

In summary, the Query–Analysis–Diagnosis Multi-Agent System works as a closed-loop reasoning process. It starts with the Query Agent analyzing potential reasons for policy failure and generating candidate problems. These problems are then answered by the Analysis Agent using detailed spatial information. Finally, the Diagnosis Agent brings everything together to identify the main cause and recommend advice to fix it. This step-by-step approach, from raising questions to finding answers and then solutions, ensures an in-depth policy failure analysis. As a result, by identifying where the policy falls short, the LLM can accurately refine the reward function, thereby accelerating its evolution.

## 4. Experiments

To validate our approach, we demonstrate FastEvo an effective framework for accelerating reward function evolution in robotic policy learning through accurate error localization and in-depth failure analysis by answering the following questions: (1) How accurately can our scoring model identify error onsets and support policy failure analysis? (2) To what extent does FastEvo accelerate the evolution of reward functions? (3) How does each component of FastEvo influence the efficiency of reward function evolution and the accuracy of error localization?

### 4.1. Scoring Model Evaluation

This section evaluates the error localization capability of our scoring model. Since we define the failure onset as the lowest-scored video frame (as in Section 3.1), we therefore go beyond overall localization accuracy to specifically assess its Policy Rollout Score Alignment and Robustness to Ambiguity Trajectories.

#### 4.1.1. Policy Rollout Score Alignment

**Baseline.** Since Visual Language Models (VLMs) can predict per-frame progress to score rollouts [28], We benchmark FastEvo against state-of-the-art VLMs and the following baselines: (1) **LIV** [26] Uses value learning on human videos for real-world value estimation; (2) **RoboCLIP** [39] Generates scores from episode-instruction encoding similarity; (3) **Video-Language Critic** [1] Assesses dense behavior-task alignment through contrastive learning and temporal ranking.

**Test Set Construction.** Our test set consists of an alignment evaluation video dataset  $\mathcal{D}_{\text{align}}$  and an error localization evaluation dataset  $\mathcal{D}_{\text{error}}$ . To construct  $\mathcal{D}_{\text{align}}$ , we sample trajectories from the Open X-Embodiment (OXE) [31] and bimanual ALOHA datasets [9]. Specifically, We randomly

| Methods               | VOC $\uparrow$ | $\rho$ $\uparrow$ | ELA $\uparrow$ |
|-----------------------|----------------|-------------------|----------------|
| LIV                   | 0.23           | 0.48              | 0.74           |
| RoboCLIP              | 0.12           | 0.02              | 0.67           |
| Video-Language Critic | 0.45           | -0.18             | 0.63           |
| GPT5 Pro              | 0.35           | 0.45              | 0.65           |
| Gemini-2.5 pro        | 0.37           | 0.42              | 0.62           |
| Claude-4.5 Sonnet     | 0.38           | 0.38              | 0.70           |
| Ours                  | <b>0.74</b>    | <b>0.84</b>       | <b>0.92</b>    |

Table 1. **Comparison of FastEvo and baselines in policy-score alignment and error localization accuracy:** We evaluate the policy-score alignment metrics (VOC,  $\rho$ ) on  $\mathcal{D}_{\text{align}}$  and the error localization accuracy (ELA) on  $\mathcal{D}_{\text{error}}$ . The onset of a policy failure is determined in two ways: by VLMs prompted to identify the initial failure frame, and by selecting the frame with the lowest score.

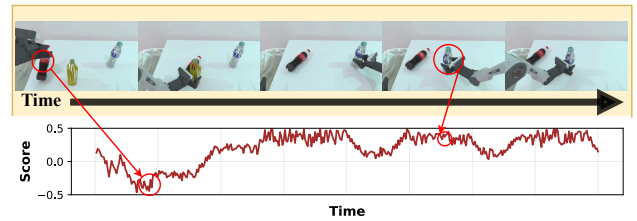


Figure 3. **Policy Score Alignment.** The results show that the scoring model assigns a low value when the robotic arm knocks over a cola bottle, indicating policy failure, and a high score when the arm successfully grasps the water bottle, demonstrating strong policy-score alignment.

select 50 trajectories from each of the 50 datasets within OXE and from the ALOHA dataset. To ensure all trajectories are semantically grounded, we use GPT-4o to automatically generate task instructions for any unannotated videos. Subsequently,  $\mathcal{D}_{\text{error}}$  is built to evaluate failure localization accuracy. It consists of: (1) **Synthetic failures:** generated by splitting each video in  $\mathcal{D}_{\text{align}}$  into four parts and shuffling the final 0%, 15%, 40%, and 70% of frames to simulate increasingly severe failure. (2) **Natural failures:** collected by deploying a DP3 model [54] in Robotwin [6], capturing realistic failure modes encountered during policy execution.

**Evaluation Metric.** Following [28], we evaluate alignment accuracy using the Video Order Consistency (VOC) metric. Additionally, we employ Spearman’s rank correlation ( $\rho$ ) to quantify the relationship between policy execution outcomes and the scores generated by our scoring model. To assess error localization capability, we define Error Localization Accuracy (ELA) as:

$$ELA = 1 - \frac{\widehat{\text{loc}}_{\text{error}} - \text{loc}_{\text{error}}}{\text{len}_{\text{traj}}} \quad (1)$$

where  $\text{len}_{\text{traj}}$  is the episode length,  $\widehat{\text{loc}}_{\text{error}}$  is the predicted failure onset location, and  $\text{loc}_{\text{error}}$  is the corresponding

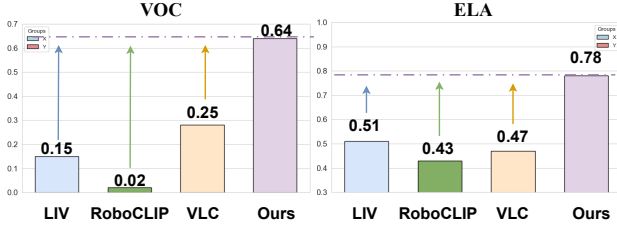


Figure 4. **Ambiguous Robustness Test.** We evaluate VOC and ELA on our test dataset. Although a performance drop in error localization is observed, the decline is considerably smaller than that of other baselines, highlighting its superior robustness.

ground-truth location.

**Result.** As shown in Table 1, our method substantially outperforms all baseline and VLM models. Notably, FastEvo exceeds the best baseline by **75.0%** and **64.4%** in policy-score alignment, with an additional gain of over **24%** in error localization accuracy. More detailed is presented in Fig. 3

#### 4.1.2. Robustness to Ambiguous Trajectories

Poorly positioned cameras in robotic policy execution often produce trajectories composed of repetitive and visually similar frames. Such ambiguous trajectories can mislead the scoring model, leading to incorrect error localization or even misclassification of successful policies as failures. To evaluate the robustness of FastEvo under these challenging conditions, we conduct an experiment focusing on ambiguous trajectories.

We first train a DP3 [54] model for each task in RoboTwin [6]. The test set is then assembled in two ways: (1) by manually selecting two visually repetitive trajectories from the rollouts for each task, and (2) by generating such trajectories from  $\mathcal{D}_{align}$  and  $\mathcal{D}_{error}$  through the insertion of duplicated video frames.

As shown in Fig. 4, FastEvo demonstrates superior performance to all baselines in both policy score alignments and error localization, highlighting its considerable robustness against visual ambiguity.

### 4.2. Reward Evolution Evaluation

As outlined in Section 3, our objective is to develop a high-quality reward function that achieves both high success rates and rapid convergence. This section accordingly evaluates the efficiency of reward function evolution, and the performance of policies trained with the reward function.

#### 4.2.1. Environment

We construct our simulation environment primarily on IsaacGym [29], integrating 10 distinct robots and 30 benchmark tasks. The task set is drawn from two sources: (1) 9 original environments provided by IsaacGym (2) 21 tasks selected from MetaWorld [52] and simulated in IsaacGym

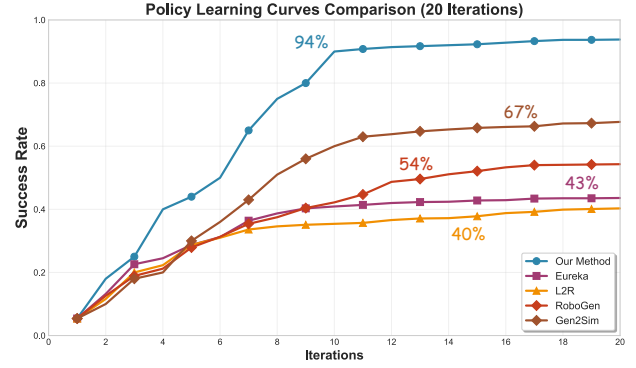


Figure 5. **Efficiency of the reward function evolution.** Evaluation across 30 tasks in IsaacGym demonstrates that our method achieves a 94% average success rate over 20 iterations, outperforming all baselines.

through the Gen2Sim framework [17]. For real-world evaluation, we deploy the Bimanual Piper robot to execute three manipulation tasks: (1) sorting blocks by color, (2) sorting blocks by size, and (3) adjusting a bottle pose. (Additional implementation details are provided in the Appendix)

#### 4.2.2. Baseline

We evaluate FastEvo against the following relevant baseline methods. (1) **Eureka** [27] iteratively improves LLM-generated rewards through GPU-accelerated evaluation. (2) **RoboGen** [44] automates robot skill acquisition by decomposing tasks, selecting suitable learning paradigms, generating supervision, and learning policies in an end-to-end framework. (3) **Gen2Sim** [17] is a framework that scales up robotic skill learning in simulation by automatically generating assets, task descriptions, and LLM-based reward functions. (4) **L2R** [53] is a two-stage LLM-prompting framework that generates reward functions by converting natural language motion descriptions into code, leveraging a set of API primitives derived from original human rewards.

#### 4.2.3. Efficiency of the reward function refinement

We evaluate the efficiency of the reward function evolution by tracking the policy’s performance at each iteration. For each iteration, a policy is trained via PPO [37] on LLM-generated rewards, utilizing Isaac Gym’s GPU-parallelized sampling, and is then evaluated over 100 rollouts to determine the task success rate. The comparison of FastEvo and all the baselines are shown in Fig. 5. Experimental results indicate that FastEvo significantly outperforms all baseline approaches in terms of average task success rate over 20 iterations, exceeding the strongest baseline (Gen2Sim) by **40%**. This confirms its capability to efficiently learn a high-quality reward function. Interestingly, the success rate occasionally shows sudden increases at specific iterations (e.g., between 9 and 10). We conclude that this phenomenon is correlated with the quality of queries provided by the query

| Module                   | (a) Error Localization |                | (b) Policy Performance             |                                    |
|--------------------------|------------------------|----------------|------------------------------------|------------------------------------|
|                          | VOC $\uparrow$         | ELA $\uparrow$ | Simulation Success rate $\uparrow$ | Real World Success rate $\uparrow$ |
| <b>Original FastEvo</b>  | <b>0.74</b>            | <b>0.92</b>    | <b>95%</b>                         | <b>78%</b>                         |
| – instruction generation | 0.08                   | 0.05           | 58%                                | 32%                                |
| – video rewinding        | 0.58                   | 0.62           | 61%                                | 38%                                |
| – demo decomposition     | 0.66                   | 0.58           | 64%                                | 44%                                |

Table 2. **Ablation Study of Scoring Model:** Performance when removing (–) various components across two evaluation metrics: (a) error localization and (b) policy performance. This study highlight the significance of data augmentation in Sec. 3.1.2.

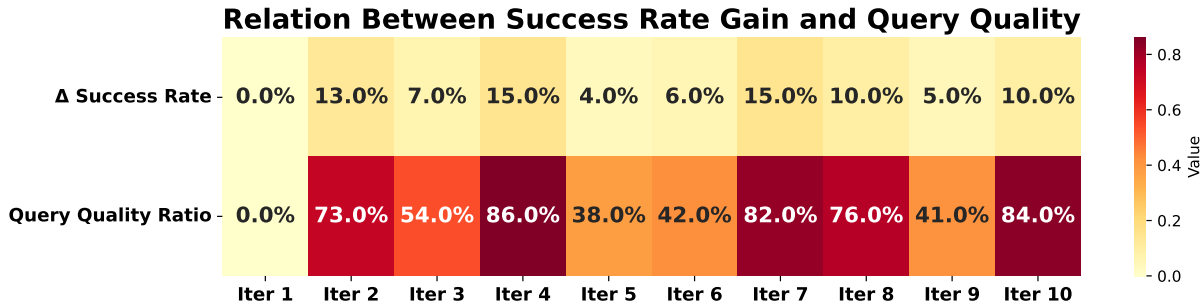


Figure 6. **Relation Between Success Rate Gain and Query Quality.** We observe a positive correlation between the growth rate of task success and the quality of queries during the early stages of reward function optimization, highlighting the critical role of the query agent within the overall framework.

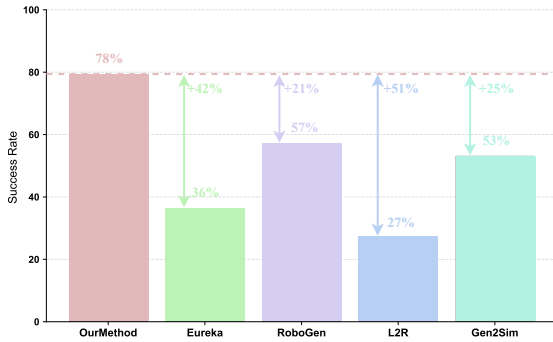


Figure 7. **Real World Policy Performance**

agent (see Sec. 3.2.1) in Fig. 6. The quality of queries is calculated as follow:

$$\text{quality} = \frac{\text{the number of root cause relevant queries}}{\text{total number of queries}}$$

#### 4.2.4. Policy Performance in Real world

We further evaluate policies trained by FastEvo and baseline methods over 50 iterations in real-world settings, using the three manipulation tasks described in Sec. 4.2.1. Results in Fig. 7 show that FastEvo achieves a higher success rate than all baseline approaches, demonstrating that its reward func-

tion evolves more efficiently throughout the learning process. (more results see Appendix)

### 4.3. Ablations

We perform an ablation study to evaluate how specific design choices in FastEvo affect error localization and reward evolution efficiency. The ablated components comprise: (1) instruction generation, video rewinding, and demo decomposition employed in scoring model training; (2) spatial features extracted from RGB video frames; (3) the query agent responsible for problem discovery; and (4) the diagnosis agent dedicated to generating actionable advice. Below, we analyze the impact of each ablation:

#### 4.3.1. Scoring Model

This section presents an ablation study on three key training components of the scoring model: instruction generation, video rewinding, and demo decomposition. We evaluate error localization performance across different component configurations to assess their individual contributions. Further, we examine how these ablated modules affect overall pipeline performance by evaluating policies trained over 20 iterations using correspondingly ablated versions of FastEvo.

**instruction generation** As shown in Sec. 4.1.2, removing instruction generation significantly degrades policy-score

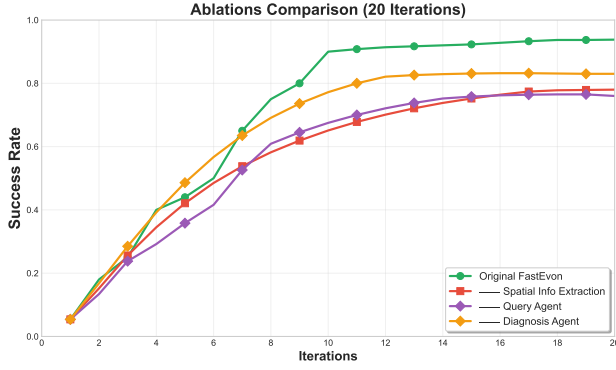


Figure 8. **Ablation Comparison.** We evaluated the contribution of each component by comparing their reward convergence curves over 20 iterations after ablating them individually.

alignment and leads to a catastrophic drop in error localization accuracy. This demonstrates that the instructions generated in data augmentation are crucial for enhancing the model’s task comprehension and error onset localization accuracy. Consequently, incorrect error localization results in low-quality diagnosis and analysis, which hinders the evolution of the reward function and ultimately leads to a discernible decline in policy performance in both simulation and real-world environments.

**video rewinding** As shown in Sec. 4.1.2(a), the ablation of video rewinding reduces both policy-score alignment and error localization accuracy, confirming its critical role in enabling the scoring model to precisely pinpoint failure onsets.

**demo decomposition** The ablation of demonstration decomposition yields an interesting outcome: while policy-score alignment experiences only a slight decline, error localization undergoes a significant degradation, with a performance drop of 34% as shown in Sec. 4.1.2. This result indicates that decomposing a global task into subtasks is crucial for enhancing the model’s precision in errors localization.

### 4.3.2. Query-Analysis-Diagnosis System

In this section, we ablate query stage, analysis stage and diagnosis stage to better understand how different design choices improve the efficiency of reward function evolution.

**Policy Performance** We conduct ablation studies on the query stage, spatial information extraction pipeline, and diagnosis agent to evaluate the policy performance trained by FastEvo under different design choices, aiming to understand their respective roles in accelerating reward function evolution (see Figure 8). Our results indicate that removing either the query agent or the diagnosis agent frequently leads the reward evolution to converge to suboptimal outcomes, and ablating the spatial information extraction module significantly slows down the evolution process.

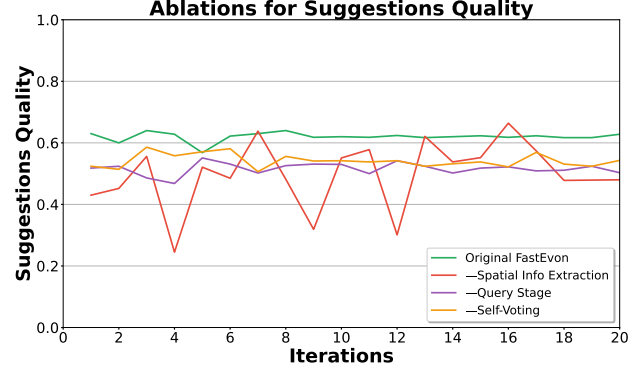


Figure 9. **Ablations for Suggestion Quality.** We ablate different key design to figure out how they impact the suggestions quality separately.

**Analysis Quality** We perform ablation studies on three key components: Spatial Information Extraction, Query Stage, and Self-Voting mechanism, to evaluate their impact on the quality of diagnostic suggestions. The quality metric is defined as:

$$\text{quality} = \frac{\text{Num}_{\text{beneficial}} - \text{Num}_{\text{harmful}}}{\text{Num}_{\text{total}}}$$

where  $\text{Num}_{\text{beneficial}}$  and  $\text{Num}_{\text{harmful}}$  represent the counts of beneficial and harmful suggestions respectively, and  $\text{Num}_{\text{total}}$  is the total number of suggestions. Results in Fig. 9 show that removing spatial information introduces high variance in suggestion quality due to the lack of spatial constraints, thereby hindering the evolution of reward function (see Fig. 8). Ablating either the query stage or self-voting mechanism consistently degrades the overall suggestion quality.

## 5. Conclusion

We propose FastEvo, a novel Query–Analysis–Diagnosis multi-agent framework built upon a precise error localization scoring model to accelerate the evolution of LLM-generated reward functions. The core strength of FastEvo lies in its well-designed collaboration mechanism among the query, analysis, and diagnosis agents, which leverages the collective capabilities of multi-agent systems to conduct in-depth policy failure analysis for efficient reward function evolution. However, FastEvo’s efficiency is constrained by the quality of Query Agent queries and the availability of spatial information. These limitations compromise the stability of the self-evolution process, driving the need for future work to enhance query generation quality and develop more robust spatial representations for improved system reliability.



## References

- [1] Minttu Alakuijala, Reginald McLean, Isaac Woungang, Nariman Farsad, Samuel Kaski, Pekka Marttinen, and Kai Yuan. Video-language critic: Transferable reward functions for language-conditioned robotics. *arXiv preprint arXiv:2405.19988*, 2024. 5
- [2] OpenAI: Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, et al. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020. 1
- [3] Apurva Badithela, David Snyder, Lihan Zha, Joseph Mikhail, Matthew O’Kelly, Anushri Dixit, and Anirudha Majumdar. Reliable and scalable robot policy evaluation with imperfect simulators. *arXiv preprint arXiv:2510.04354*, 2025. 2
- [4] Berk Calli, Aaron Walsman, Arjun Singh, Siddhartha Srinivasa, Pieter Abbeel, and Aaron M Dollar. Benchmarking in manipulation research: The ycb object and model set and benchmarking protocols. *arXiv preprint arXiv:1502.03143*, 2015. 2
- [5] Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. Chateval: Towards better llm-based evaluators through multi-agent debate. *arXiv preprint arXiv:2308.07201*, 2023. 1, 2
- [6] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025. 1, 2, 5, 6
- [7] Ali Dorri, Salil S Kanhere, and Raja Jurdak. Multi-agent systems: A survey. *Ieee Access*, 6:28573–28593, 2018. 2
- [8] Iddo Drori and Dov Te’eni. Human-in-the-loop ai reviewing: feasibility, opportunities, and risks. *Journal of the Association for Information Systems*, 25(1):98–109, 2024. 1
- [9] Zipeng Fu, Tony Z. Zhao, and Chelsea Finn. Mobile aloha: Learning bimanual mobile manipulation with low-cost whole-body teleoperation, 2024. 5
- [10] Milan Groshev, Javier Sacido, and Jorge Martín-Pérez. Foreco: a forecast-based recovery mechanism for real-time remote control of robotic manipulators. In *Proceedings of the SIGCOMM’22 Poster and Demo Sessions*, pages 7–9. 2022. 2
- [11] Songhao Han, Boxiang Qiu, Yue Liao, Siyuan Huang, Chen Gao, Shuicheng Yan, and Si Liu. Robocerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation. *arXiv preprint arXiv:2506.06677*, 2025. 2
- [12] Ankur Handa, Arthur Allshire, Viktor Makoviychuk, Aleksei Petrenko, Ritvik Singh, Jingzhou Liu, Denys Makoviichuk, Karl Van Wyk, Alexander Zhurkevich, Balakumar Sundaralingam, et al. Dextreme: Transfer of agile in-hand manipulation from simulation to reality. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5977–5984. IEEE, 2023. 1
- [13] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2023. 1, 2
- [14] Siyuan Huang, Zhengkai Jiang, Hao Dong, Yu Qiao, Peng Gao, and Hongsheng Li. Instruct2act: Mapping multimodality instructions to robotic actions with large language model. *arXiv preprint arXiv:2305.11176*, 2023. 2
- [15] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023. 2
- [16] Yuheng Ji, Huajie Tan, Jiayu Shi, Xiaoshuai Hao, Yuan Zhang, Hengyuan Zhang, Pengwei Wang, Mengdi Zhao, Yao Mu, Pengju An, et al. Robobrain: A unified brain model for robotic manipulation from abstract to concrete. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1724–1734, 2025. 2
- [17] Pushkal Katara, Zhou Xian, and Katerina Fragkiadaki. Gen2sim: Scaling up robot learning in simulation with generative models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6672–6679. IEEE, 2024. 2, 6
- [18] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik S Chan, Xuhai Xu, Daniel McDuff, Hyeonhoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae W Park. Mdagents: An adaptive collaboration of llms for medical decision-making. *Advances in Neural Information Processing Systems*, 37: 79410–79452, 2024. 2
- [19] Zhiqian Lan, Yuxuan Jiang, Ruiqi Wang, Xuanbing Xie, Rongkui Zhang, Yicheng Zhu, Peihang Li, Tianshuo Yang, Tianxing Chen, Haoyu Gao, et al. Autobio: A simulation and benchmark for robotic automation in digital biology laboratory. *arXiv preprint arXiv:2505.14030*, 2025. 2
- [20] Bingxuan Li, Yiwei Wang, Jiuxiang Gu, Kai-Wei Chang, and Nanyun Peng. Metal: A multi-agent framework for chart generation with test-time scaling. *arXiv preprint arXiv:2502.17651*, 2025. 1, 2
- [21] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, et al. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024. 2
- [22] Yaxuan Li, Yichen Zhu, Junjie Wen, Chaomin Shen, and Yi Xu. Worlddeval: World model as real-world robot policies evaluator. *arXiv preprint arXiv:2505.19017*, 2025. 1, 2
- [23] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022. 2
- [24] Kevin Lin, Christopher Agia, Toki Migimatsu, Marco Pavone, and Jeannette Bohg. Text2motion: From natural language instructions to feasible plans. *Autonomous Robots*, 47(8):1345–1365, 2023. 2
- [25] Ziyuan Liu, Wei Liu, Yuzhe Qin, Fanbo Xiang, Minghao Gou, Songyan Xin, Maximo A Roa, Berk Calli, Hao Su, Yu Sun, et al. Ocrto: A cloud-based competition and bench-

- mark for robotic grasping and manipulation. *IEEE Robotics and Automation Letters*, 7(1):486–493, 2021. 2
- [26] Yecheng Jason Ma, Vikash Kumar, Amy Zhang, Osbert Bastani, and Dinesh Jayaraman. Liv: Language-image representations and rewards for robotic control. In *International Conference on Machine Learning*, pages 23301–23320. PMLR, 2023. 5
- [27] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023. 1, 2, 6
- [28] Yecheng Jason Ma, Joey Hejna, Chuyuan Fu, Dhruv Shah, Jacky Liang, Zhuo Xu, Sean Kirmani, Peng Xu, Danny Driess, Ted Xiao, et al. Vision language models are in-context value learners. In *The Thirteenth International Conference on Learning Representations*, 2024. 5
- [29] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, et al. Isaac gym: High performance gpu-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021. 1, 6
- [30] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023. 4
- [31] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024. 2, 4, 5
- [32] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023. 2
- [33] Julian Quevedo, Percy Liang, and Sherry Yang. Evaluating robot policies in a world model. *arXiv preprint arXiv:2506.00613*, 2025. 1, 2
- [34] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. 4
- [35] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019. 4
- [36] Stuart Russell, Peter Norvig, and Artificial Intelligence. A modern approach. *Artificial Intelligence. Prentice-Hall, Englewood Cliffs*, 25(27):79–80, 1995. 1
- [37] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 6
- [38] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*, 2022. 2
- [39] Sumedh Sontakke, Jesse Zhang, Séb Arnold, Karl Pertsch, Erdem Bıyık, Dorsa Sadigh, Chelsea Finn, and Laurent Itti. Roboclip: One demonstration is enough to learn robot policies. *Advances in Neural Information Processing Systems*, 36:55681–55693, 2023. 5
- [40] Peng Su, Kun Wang, Xingyu Zeng, Shixiang Tang, Dapeng Chen, Di Qiu, and Xiaogang Wang. Adapting object detectors with conditional domain normalization. In *European Conference on Computer Vision*, pages 403–419. Springer, 2020. 1, 2
- [41] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, USA, second edition, 2018. 1
- [42] Huajie Tan, Xiaoshuai Hao, Cheng Chi, Minglan Lin, Yaoxu Lyu, Mingyu Cao, Dong Liang, Zhuo Chen, Mengsi Lyu, Cheng Peng, et al. Roboos: A hierarchical embodied framework for cross-embodiment and multi-agent collaboration. *arXiv preprint arXiv:2505.03673*, 2025. 2
- [43] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. 2
- [44] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via generative simulation. *arXiv preprint arXiv:2311.01455*, 2023. 1, 2, 6
- [45] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. 2
- [46] Anxing Xiao, Nuwan Janaka, Tianrun Hu, Anshul Gupta, Kaixin Li, Cunjun Yu, and David Hsu. Robi butler: Multimodal remote interaction with a household robot assistant. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4337–4344. IEEE, 2025. 2
- [47] Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2reward: Reward shaping with language models for reinforcement learning. *arXiv preprint arXiv:2309.11489*, 2023. 1, 2
- [48] Adina Yakefu, Bin Xie, Chongyang Xu, Enwen Zhang, Erjin Zhou, Fan Jia, Haitao Yang, Haoqiang Fan, Haowei Zhang, Hongyang Peng, et al. Robochallenge: Large-scale real-robot evaluation of embodied policies. *arXiv preprint arXiv:2510.17950*, 2025. 2
- [49] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *Advances in Neural Information Processing Systems*, 37:21875–21911, 2024. 4
- [50] Xuning Yang, Clemens Eppner, Jonathan Tremblay, Dieter Fox, Stan Birchfield, and Fabio Ramos. Robot policy evaluation for sim-to-real transfer: A benchmarking perspective. *arXiv preprint arXiv:2508.11117*, 2025. 2

- 725 [51] Weirui Ye, Fangchen Liu, Zheng Ding, Yang Gao, Oleh Ry-  
726 bkin, and Pieter Abbeel. Video2policy: Scaling up manip-  
727 ulation tasks in simulation through internet videos. *arXiv*  
728 *preprint arXiv:2502.09886*, 2025. 2
- 729 [52] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian,  
730 Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-  
731 world: A benchmark and evaluation for multi-task and meta  
732 reinforcement learning. In *Conference on robot learning*,  
733 pages 1094–1100. PMLR, 2020. 6
- 734 [53] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani,  
735 Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis  
736 Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, et al.  
737 Language to rewards for robotic skill synthesis. *arXiv*  
738 *preprint arXiv:2306.08647*, 2023. 1, 2, 6
- 739 [54] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu,  
740 Muhan Wang, and Huazhe Xu. 3d diffusion policy: Gen-  
741 eralizable visuomotor policy learning via simple 3d repre-  
742 sentations. *arXiv preprint arXiv:2403.03954*, 2024. 2, 4, 5,  
743 6
- 744 [55] Jiahui Zhang, Yusen Luo, Abrar Anwar, Sumedh Anand  
745 Sontakke, Joseph J Lim, Jesse Thomason, Erdem Biyik,  
746 and Jesse Zhang. Rewind: Language-guided rewards teach  
747 robot policies without new demonstrations. *arXiv preprint*  
748 *arXiv:2505.10911*, 2025. 3, 4
- 749 [56] Yang Zhang, Shixin Yang, Chenjia Bai, Fei Wu, Xiu Li,  
750 Zhen Wang, and Xuelong Li. Towards efficient llm ground-  
751 ing for embodied multi-agent collaboration. *arXiv preprint*  
752 *arXiv:2405.14314*, 2024. 2
- 753 [57] Zhiyuan Zhou, Pranav Atreya, You Liang Tan, Karl Pertsch,  
754 and Sergey Levine. Autoeval: Autonomous evaluation of  
755 generalist robot manipulation policies in the real world.  
756 *arXiv preprint arXiv:2503.24278*, 2025. 1, 2